

Single-Cycle MIPS Processor

Three-page interview guide based on the modules in your MIPS project

1. Big Picture: What You Built

You built a small 32-bit single-cycle MIPS processor. Every instruction completes in one clock cycle. The processor is split into a control unit, a datapath, instruction memory, and data memory.

Block	Job	Your file
top	Connects the processor and both memories	top.v
mips	Combines controller and datapath	mips.v
controller	Translates instruction fields into control signals	controller.v
datapath	Moves data, calculates addresses, and updates PC	datapath.v
imem	Returns the instruction selected by PC	imem.v
dmem	Reads and writes program data	dmem.v

Top-level data flow

```
PC -> imem -> instr -> mips
mips -> dataaddr, writedata, memwrite -> dmem
dmem -> readdata -> mips
```

The PC is an address, not an instruction. The processor generates the PC. Instruction memory uses it to choose a 32-bit instruction and returns that instruction on instr.

2. What Happens During One Cycle

Step	Hardware action
1. Fetch	PC selects an instruction in imem. The instruction is returned as instr.
2. Decode	The controller reads opcode instr[31:26] and funct instr[5:0]. The register file reads rs and rt.
3. Execute	The ALU performs add, subtract, AND, OR, or SLT.
4. Memory	lw reads dmem; sw writes dmem; other instructions do not access data memory.
5. Write-back	The ALU result or loaded memory data is written to one register when regwrite is 1.
6. Next PC	The PC register captures pcnext on the rising clock edge.

Important address detail

MIPS instructions are 4 bytes, so PC values normally increase by 4. imem receives pc[7:2], which removes the two byte-offset bits: PC 0 selects instruction 0, PC 4 selects instruction 1, and PC 8 selects instruction 2.

3. Datapath and Control Signals

The controller makes decisions; the datapath performs the operations. The following signals are the central interview vocabulary.

Signal	Meaning
regwrite	1 writes a result into the register file.
regdst	0 chooses rt as destination; 1 chooses rd.
alusrc	0 selects the second register value; 1 selects the sign-extended immediate.
memtoreg	0 writes the ALU result; 1 writes data read from dmem.
memwrite	1 enables a data-memory write for sw.
branch	1 says the instruction is a conditional branch.
pcsrc	branch & zero; selects the branch target when beq is true.
jump	Selects the jump target instead of the sequential or branch PC.
aluop	Broad ALU category: 00 add, 01 subtract, 10 inspect funct.
alucontrol	Exact ALU command: 000 AND, 001 OR, 010 ADD, 110 SUB, 111 SLT.
zero	ALU result is zero; used by beq equality checking.

4. Instruction Types

Instruction	ALU work	Key controls
R-type: add, sub, and, or, slt	Two register operands	regwrite=1, regdst=1, alusrc=0, aluop=10
lw	Base register + immediate address	alusrc=1, memtoreg=1, regwrite=1, aluop=00
sw	Base register + immediate address	alusrc=1, memwrite=1, regwrite=0, aluop=00
beq	Subtract registers to compare	branch=1, aluop=01; take if zero=1
addi	Register + sign-extended immediate	alusrc=1, regwrite=1, regdst=0, aluop=00
j	No ALU data operation required	jump=1

5. PC Selection

```
pcplus4 = pc + 4
pcbranch = pcplus4 + (signimm << 2)
pcnextbr = pcsrc ? pcbranch : pcplus4
pcnext = jump ? jump_target : pcnextbr
```

For beq, the ALU subtracts the two register values. If the result is zero, zero=1. Since pcsrc=branch & zero, only a true beq condition selects the branch target.

6. Follow One Real Program

Your memfile.dat contains hexadecimal machine instructions. imem loads them with \$readmemh. One line is one 32-bit instruction.

```
20020005    addi $2, $0, 5
2003000c    addi $3, $0, 12
2067fff7    addi $7, $3, -9
00e22025    or   $4, $7, $2
00642824    and  $5, $3, $4
00a42820    add  $5, $5, $4
ac670044    sw   $7, 68($3)
8c020050    lw   $2, 80($0)
ac020054    sw   $2, 84($0)
```

The final store succeeds because register \$2 receives 7 from memory address 80, then sw calculates 0 + 84 and writes that value to address 84.

```
$3 = 12
$7 = 12 + (-9) = 3
$4 = $7 OR $2 = 3 OR 5 = 7
store at 80 receives 7
load from 80 gives $2 = 7
store at 84 writes 7
```

7. Interview Answers You Can Use

Question	Short answer
Why single-cycle?	It is simple to understand: one instruction completes per clock cycle, although the clock must be long enough for the slowest instruction.
Why two register reads?	R-type and beq instructions need two operands at once. There is one write port because an instruction normally writes one destination.
What does aluop do?	It gives the ALU decoder a broad category. lw/sw need add, beq needs subtract, and R-type uses funct for the exact operation.
What is pccsrc?	It selects the next PC source. pccsrc=0 uses PC+4; pccsrc=1 uses the branch target.
Why shift branch immediate by 2?	The immediate counts instructions, while addresses count bytes. Multiplying by 4 converts instruction offset to byte offset.
What is register zero?	Register 0 always reads as zero. It is useful for constants and base addresses, such as sw \$2,84(\$0).

8. How to Demonstrate It

```
cd /home/anton/four_digit_stopwatch
make clean
make run
```

Expected output:

Simulation succeeded

If asked about verification, explain that testbench.v drives clock and reset, imem loads memfile.dat, and the testbench watches dataaddr, writedata, and memwrite. It declares success when the processor performs the expected store of 7 to address 84.

One honest limitation to mention: a single-cycle design is educational and clear, but it is slower than a pipelined design because every instruction uses one long clock period.

9. Complete Test Program: Assembly to Machine Code

The address column is the byte address of the instruction. The machine column is the 32-bit hexadecimal value loaded into imem. Because each instruction is 4 bytes, addresses increase by 4.

Label	Assembly	Meaning	Addresses	Machine code
main	addi \$2,\$0,5	initialize \$2 = 5	0	20020005
	addi \$3,\$0,12	initialize \$3 = 12	4	2003000c
	addi \$7,\$3,-9	$\$7 = 12 - 9 = 3$	8	2067fff7
	or \$4,\$7,\$2	$\$4 = 3 \text{ OR } 5 = 7$	12	00e22025
	and \$5,\$3,\$4	$\$5 = 12 \text{ AND } 7 = 4$	16	00642824
	add \$5,\$5,\$4	$\$5 = 4 + 7 = 11$	20	00a42820
	beq \$5,\$7,end	$11 \neq 3$, branch not taken	24	10a7000a
	slt \$4,\$3,\$4	$12 < 7$ is false, $\$4 = 0$	28	0084202a
	beq \$4,\$0,around	$0 == 0$, branch taken	32	10800001
	addi \$5,\$0,0	skipped by branch	36	20050000
around	slt \$4,\$7,\$2	$3 < 5$ is true, $\$4 = 1$	40	00e2202a
	add \$7,\$4,\$5	$\$7 = 1 + 11 = 12$	44	00853820
	sub \$7,\$7,\$2	$\$7 = 12 - 5 = 7$	48	00e23822
	sw \$7,68(\$3)	memory[80] = 7	52	ac670044
	lw \$2,80(\$0)	$\$2 = \text{memory}[80] = 7$	56	8c020050
	j end	jump over addi	60	08000011
	addi \$2,\$0,1	skipped by jump	64	20020001
end	sw \$2,84(\$0)	final store	68	ac020054

Note

The corrected project memfile.dat uses 00853820 for add \$7,\$4,\$5 and 10000001 for the unconditional beq \$0,\$0,around. Those values must match the assembly program used by the testbench.

10. How to Verify the Processor

Verification means proving that the hardware produces the expected behavior, not merely proving that the Verilog compiles.

Step 1: Compile all RTL

```
cd /home/anton/four_digit_stopwatch
make clean
make compile
```

Compilation checks syntax and creates the work library. Every referenced module must be included in RTL_SRCS or compiled by the wildcard command.

Step 2: Run the automated testbench

```
make run
```

The testbench drives reset and clock. imem loads memfile.dat. The testbench watches memwrite, dataaddr, and writedata. It prints Simulation succeeded when a store writes 7 to address 84.

Step 3: Understand the expected final transaction

Signal	Expected value	Reason
memwrite	1	The instruction is sw.
dataaddr	84	$\$0 + \text{immediate } 84 = 84$.
writedata	7	$\$2$ was loaded from memory location 80.
dmem index	21	Word index is $\text{address}[31:2] = 84 / 4$.

Step 4: Verify interactively with Questa

```
vlog -sv MIPS/*.v
vsim -gui -voptargs="+acc" work.testbench
add wave -r /*
run -all
```

In the waveform, inspect pc, instr, dataaddr, writedata, memwrite, and the register-file values. At the successful final store, pc is 68, instr is ac020054, memwrite is 1, dataaddr is 84, and writedata is 7.

Step 5: Verify the memory update

```
vsim -c -voptargs="+acc" work.testbench
run 175ns
examine -radix decimal /testbench/dut/dataaddr
examine -radix decimal /testbench/dut/writedata
examine /testbench/dut/memwrite
examine -radix decimal /testbench/dut/dmem/RAM(21)
```

RAM[21] corresponds to byte address 84. After the final rising edge, it should contain 7. This is the strongest direct check that dmem received and stored the correct transaction.

Interview summary

Say: I implemented and verified a 32-bit educational single-cycle MIPS processor. The PC fetches instructions from imem, the controller decodes the opcode and funct fields, the datapath executes the operation, dmem handles loads and stores, and a self-checking testbench confirms the final store transaction.